Universidad del Valle de Guatemala

Bases de datos 1

Proyecto 2

David López - 24730

Descripción Técnica

Descripción del sistema y alcance

El sistema propuesto consiste en una aplicación web diseñada para administrar el inventario y las ventas de Bubu's Bakery, una repostería boutique guatemalteca especializada en brownies artesanales. Actualmente, muchas reposterías de barrio manejan su inventario con cuadernos físicos o con hojas de cálculo dispersas, lo que dificulta llevar el control real del stock, identificar los productos más vendidos y conocer el comportamiento de las ventas por sucursal.

La plataforma busca centralizar este proceso, permitiendo que la administradora cuente con un único panel desde el cual puede registrar productos, gestionar clientes, registrar ventas con cálculo automático de impuestos, y consultar reportes analíticos sobre el desempeño del negocio. Cada operación de venta se ejecuta dentro de una transacción explícita que actualiza simultáneamente la cabecera de la venta, sus detalles, el stock de los productos involucrados y la bitácora de movimientos de inventario.

Dentro del sistema existe un único tipo de usuario funcional: la administradora de la cadena. Sin embargo, la base de datos modela 25 empleados de distintos roles (pasteleros, cajeros, baristas, vendedores, etc.) distribuidos en 25 sucursales, ya que estos registros son referenciados por las ventas y compras como "responsable" o "operador". El acceso a la aplicación se controla mediante autenticación con sesión y contraseñas hasheadas con bcrypt.

El sistema incluye funcionalidades de soporte como el cálculo automático del IVA (12%), generación de número de factura correlativa, validación en línea del stock disponible al momento de registrar una venta, exportación de reportes a CSV y un mecanismo de soft delete para productos y clientes que preserva la integridad referencial del histórico de ventas.

El alcance del sistema se enfoca en la operación diaria del negocio: catálogo de productos, alta/edición/baja de clientes, registro de ventas con detalle, control de stock con bitácora, y reportes para la toma de decisiones. No se contempla en esta versión la gestión de cuentas bancarias, la facturación electrónica con SAT, ni la integración con plataformas de delivery externas.

Identificación formal de entidades y atributos

El modelo está organizado en 6 bloques temáticos que agrupan las 14 entidades del sistema.

Bloque 1 — Catálogo

- Categoria: nombre, descripción.
- Marca: nombre, descripción.
- Producto: SKU, nombre, descripción, precio, stock, stock mínimo, activo, fecha de creación.

Bloque 2 — Personas

- Rol: nombre, descripción.
- Empleado: nombres, apellidos, email, teléfono, username, password hash (bcrypt), activo, fecha de ingreso.
- Proveedor: nombre, NIT, teléfono, email, dirección, activo.
- Cliente: nombres, apellidos, NIT, teléfono, email, dirección, fecha de registro, activo.

Bloque 3 — Infraestructura

- Sucursal: nombre, dirección, teléfono, activa.
- MetodoPago: nombre, activo.

Bloque 4 — Compras a proveedor

- CompraProveedor: fecha, número de factura, subtotal, impuesto, total, estado (COMPLETADA / ANULADA).
- DetalleCompra: cantidad, costo unitario, subtotal.

Bloque 5 — Ventas a cliente

- Venta: fecha, número de factura, subtotal, impuesto, total, estado (COMPLETADA / ANULADA).
- DetalleVenta: cantidad, precio unitario (snapshot al momento de la venta), subtotal.

Bloque 6 — Trazabilidad

- MovimientoStock: fecha, tipo (ENTRADA / SALIDA / AJUSTE), cantidad, stock resultante, motivo.

Identificación de claves candidatas y clave primaria seleccionada

Categoria

- Llaves candidatas: id_categoria, nombre.
- Clave primaria seleccionada: id_categoria.

Marca

- Llaves candidatas: id_marca, nombre.
- Clave primaria seleccionada: id_marca.

Producto

- Llaves candidatas: id_producto, sku.
- Clave primaria seleccionada: id_producto.

Rol

- Llaves candidatas: id_rol, nombre.
- Clave primaria seleccionada: id_rol.

Sucursal

- Llaves candidatas: id_sucursal, nombre.
- Clave primaria seleccionada: id_sucursal.

Empleado

- Llaves candidatas: id_empleado, email, username.
- Clave primaria seleccionada: id_empleado.

Proveedor

- Llaves candidatas: id_proveedor, NIT.
- Clave primaria seleccionada: id_proveedor.

Cliente

- Llaves candidatas: id_cliente, NIT.
- Clave primaria seleccionada: id_cliente (NIT no aplica como PK porque puede ser NULL en ventas de mostrador / Consumidor Final).

MetodoPago

- Llaves candidatas: id_metodo_pago, nombre.
- Clave primaria seleccionada: id_metodo_pago.

CompraProveedor

- Llaves candidatas: id_compra.
- Clave primaria seleccionada: id_compra.

DetalleCompra

- Llaves candidatas: id_detalle_compra, (id_compra, id_producto).
- Clave primaria seleccionada: id_detalle_compra (existe constraint UNIQUE(id_compra, id_producto) para evitar duplicados de un mismo producto en una misma compra).

Venta

- Llaves candidatas: id_venta, numero_factura.
- Clave primaria seleccionada: id_venta.

DetalleVenta

- Llaves candidatas: id_detalle_venta, (id_venta, id_producto).
- Clave primaria seleccionada: id_detalle_venta (con UNIQUE(id_venta, id_producto)).

MovimientoStock

- Llaves candidatas: id_movimiento.
- Clave primaria seleccionada: id_movimiento.

Dependencias funcionales claramente definidas

Catálogo

- id_categoria → nombre, descripción.
- id_marca → nombre, descripción.
- id_producto → sku, nombre, descripción, precio, stock, stock_minimo, activo, fecha_creacion, id_categoria, id_marca.
- sku → id_producto (sku es candidata, equivalente).

Personas

- id_rol → nombre, descripción.
- id_empleado → nombres, apellidos, email, telefono, username, password_hash, activo, fecha_ingreso, id_rol, id_sucursal.
- email → id_empleado (única).
- username → id_empleado (única).
- id_proveedor → nombre, NIT, telefono, email, direccion, activo.
- NIT → id_proveedor (cuando NIT no es nulo).
- id_cliente → nombres, apellidos, NIT, telefono, email, direccion, fecha_registro, activo.

Infraestructura

- id_sucursal → nombre, direccion, telefono, activa.
- id_metodo_pago → nombre, activo.

Compras

- id_compra → fecha, numero_factura, subtotal, impuesto, total, estado, id_proveedor, id_empleado, id_sucursal.
- id_detalle_compra → cantidad, costo_unitario, subtotal, id_compra, id_producto.
- (id_compra, id_producto) → id_detalle_compra (UNIQUE).

Ventas

- id_venta → fecha, numero_factura, subtotal, impuesto, total, estado, id_cliente, id_empleado, id_sucursal, id_metodo_pago.
- id_detalle_venta → cantidad, precio_unitario, subtotal, id_venta, id_producto.
- (id_venta, id_producto) → id_detalle_venta (UNIQUE).

Trazabilidad

- id_movimiento → fecha, tipo, cantidad, stock_resultante, motivo, id_producto, id_empleado, id_sucursal, id_compra, id_venta.

Normalización justificada hasta 3FN

Diseño no normalizado

Si modeláramos toda la información del negocio en una sola tabla "operación", caeríamos en una estructura como la siguiente:

```
Operacion(numero_factura, fecha, tipo,
    producto_sku, producto_nombre, producto_categoria, producto_marca,
    cliente_nombre, cliente_nit,
    empleado_nombre, empleado_rol, empleado_sucursal,
    metodo_pago, cantidad, precio_unitario, subtotal, impuesto, total)
```

Este diseño presenta varios problemas que la normalización resuelve.

- Aplicación de 1FN (Primera Forma Normal)

Regla: todos los atributos deben ser atómicos (un valor por celda) y no deben existir grupos repetidos.

Violación detectada en el diseño no normalizado: una venta puede incluir varios productos. En el diseño plano esto obligaría a tener columnas como producto1_sku, producto2_sku, producto3_sku, ... o a guardar listas dentro de una celda (por ejemplo "BRW-0001, BRW-0002"). Ambos enfoques rompen 1FN.

Solución aplicada: se descompone la entidad "Operación" en una cabecera y un detalle:

- Venta(id_venta, fecha, numero_factura, subtotal, impuesto, total, estado, ...)
- DetalleVenta(id_detalle_venta, id_venta, id_producto, cantidad, precio_unitario, subtotal)

La misma técnica se aplica para las compras a proveedor:

- CompraProveedor(id_compra, fecha, numero_factura, ...)
- DetalleCompra(id_detalle_compra, id_compra, id_producto, cantidad, costo_unitario, subtotal)

Ahora, una venta con N productos genera 1 fila en Venta y N filas en DetalleVenta. Cada celda contiene un valor atómico.

- Aplicación de 2FN (Segunda Forma Normal)

Regla: estar en 1FN y, además, todo atributo no clave debe depender de la clave primaria completa, no solo de una parte de ella.

Violación detectada (potencial): si la PK de DetalleVenta fuera la combinación (id_venta, id_producto), atributos como precio_unitario o subtotal dependen de la combinación completa, mientras que un atributo como nombre_producto (si lo hubiéramos guardado allí) dependería solo de id_producto — violando 2FN.

Solución aplicada:

- Se usa una clave primaria simple (id_detalle_venta) y se mantiene la combinación (id_venta, id_producto) como UNIQUE.
- No se duplican atributos del producto en el detalle: nombre_producto no existe en DetalleVenta, se obtiene siempre por JOIN a Producto.
- La única excepción es precio_unitario, que sí se almacena intencionalmente como snapshot. Esto se discute en la sección de denormalización justificada al final.

Como todas las claves primarias del proyecto son simples (todos los id_* son SERIAL, columnas únicas), 2FN se cumple de forma automática en todas las tablas.

- Aplicación de 3FN (Tercera Forma Normal)

Regla: estar en 2FN y, además, no debe existir ninguna dependencia transitiva — es decir, ningún atributo no clave debe depender de otro atributo no clave.

Violaciones detectadas y solucionadas:

a) Categoría y Marca como tablas separadas

En el diseño no normalizado, Producto tendría columnas como categoria_nombre y marca_nombre directamente. Esto introduce dos dependencias transitivas:

id_producto → categoria_nombre
id_producto → marca_nombre

Pero el categoria_nombre no depende realmente del producto: depende de qué categoría es. Si dos productos pertenecen a la categoría "Brownies Clásicos" y queremos renombrarla a "Brownies Tradicionales", tendríamos que actualizar todas las filas de Producto que usan esa categoría — anomalía de actualización clásica.

Solución: se extraen Categoria y Marca a tablas propias. Producto solo guarda id_categoria y id_marca como FKs. Renombrar una categoría es ahora un único UPDATE Categoria SET nombre = … WHERE id_categoria = ….

b) Rol como tabla separada

Originalmente, Empleado podría tener una columna rol VARCHAR con valores como 'Cajero', 'Pastelero Jefe', etc. Esto introduciría:

id_empleado → rol_nombre, rol_descripcion

Pero rol_descripcion depende de rol_nombre, no del empleado. Misma anomalía de actualización.

Solución: se extrae Rol a tabla propia. Empleado solo guarda id_rol.

c) Sucursal y MetodoPago como tablas separadas

El mismo razonamiento aplica para Sucursal (referenciada por Empleado, Venta, CompraProveedor, MovimientoStock) y para MetodoPago (referenciado por Venta). Si guardáramos nombre_sucursal o nombre_metodo_pago directamente en cada operación, el nombre de la sucursal dependería transitivamente de cuál sucursal es, no de cuál venta es.

Solución: se extraen ambas a tablas catálogo.

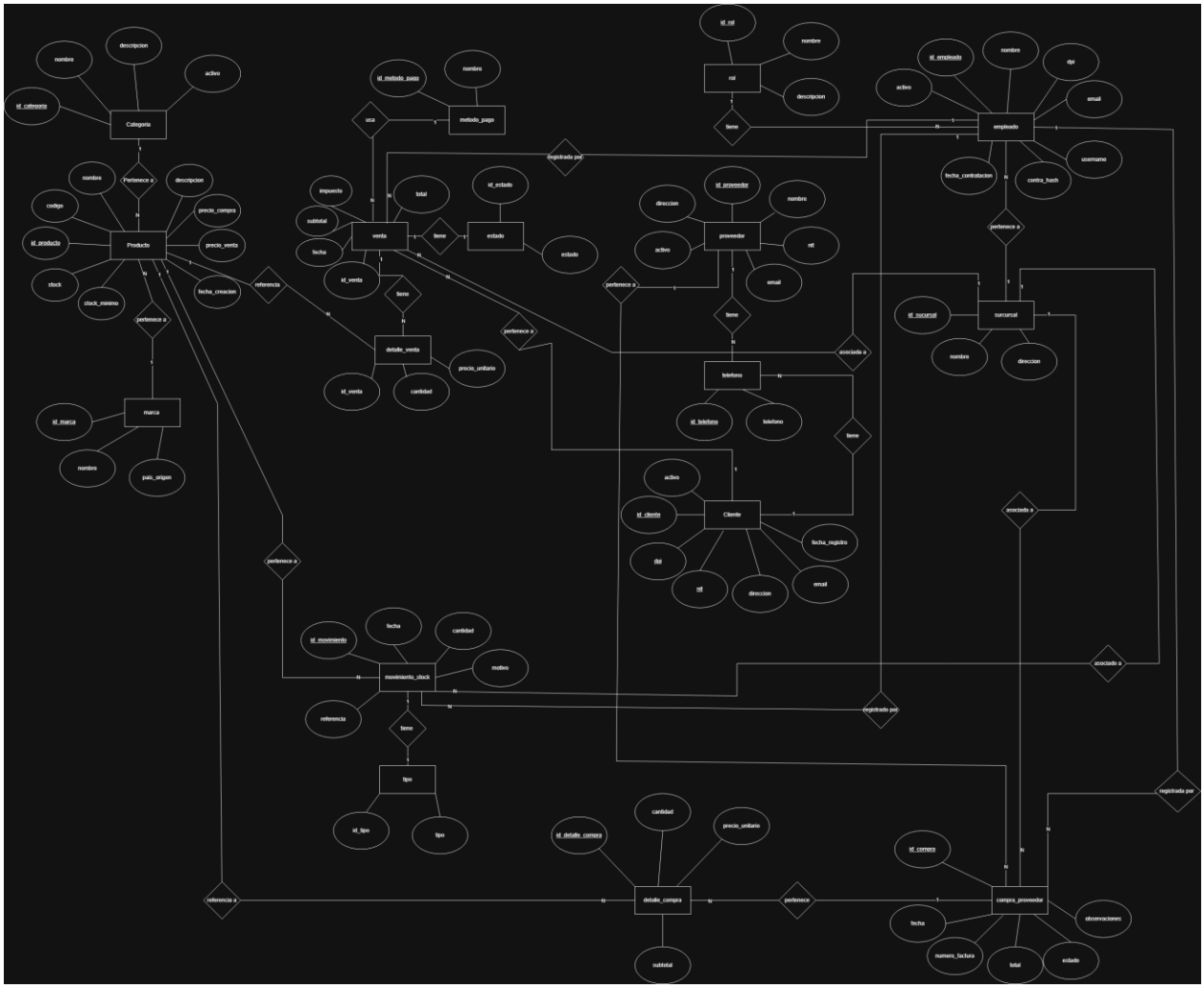
d) MovimientoStock con referencia polimórfica controlada

MovimientoStock registra entradas (por compra), salidas (por venta) y ajustes manuales. Una entrada exige saber la compra de origen; una salida exige saber la venta. Para no duplicar columnas como tipo_origen, id_origen, se modelan dos FKs nullable (id_compra, id_venta) más un constraint CHECK que garantiza la consistencia:

```
CHECK (
  (tipo = 'ENTRADA' AND id_compra IS NOT NULL AND id_venta IS NULL) OR
  (tipo = 'SALIDA'  AND id_venta  IS NOT NULL AND id_compra IS NULL) OR
  (tipo = 'AJUSTE'  AND id_compra IS NULL     AND id_venta  IS NULL)
)
```

Esto se conoce como exclusive arc y permite que la integridad de las referencias quede a nivel de la base, no de la aplicación. La tabla queda en 3FN porque cada atributo no-clave depende solo del id_movimiento.

DER


id_rol
nombre
rol
descripcion
tiene
id_empleado
nombre
dpi
activo
email
empleado
usuario
fecha_contratacion
contra_hash
pertenece a
id_sucursal
sucursal
nombre
direccion
id_categoria
nombre
descripcion
activo
Categoria
id_metodo_pago
nombre
metodo_pago
usa
registrada por
nombre
Pertenece a
descripcion
codigo
precio_compra
id_producto
Producto
precio_venta
stock
stock_minimo
fecha_creacion
referencia
pertenece a
impuesto
total
subtotal
venta
tiene
id_estado
estado
estado
fecha
id_venta
tiene
detalle_venta
precio_unitario
id_venta
cantidad
id_proveedor
direccion
nombre
proveedor
activo
rif
email
pertenece a
tiene
asociada a
telefono
id_telefono
telefono
tiene
id_marca
marca
nombre
pais_origen
activo
id_cliente
Cliente
fecha_registro
dpi
rif
direccion
email
asociada a
pertenece a
id_movimiento
fecha
cantidad
motivo
movimiento_stock
referencia
tiene
registrado por
asociado a
tipo
id_tipo
tipo
registrada por
id_detalle_compra
cantidad
precio_unitario
id_compra
referencia a
detalle_compra
pertenece
compra_proveedor
fecha
numero_factura
total
estado
observaciones
subtotal